Generative Modelling Case Study

A Practical Exploration of Generative Adversarial Networks

ID:24064958

Github: https://github.com/Ravikumar0604/Generative_modeling_casestudy

Table of Contents

1. Introduction 2
2. Part 1GANs on Synthetic 2D Data 2
2.1 Sine Wave GAN 2
2.2 Spiral Distribution GAN 3
2.3 Architecture Comparison 3
3. Part 2.1OCTMNIST Retinal OCT Images 4
3.1 Dataset 4
3.2 DCGAN Architecture 4
3.3 FID Evaluation 4
3.4 ExtensionConditional GAN 5
3.5 DiscussionGANs for Generating Ducks 5
4. Part 2.2CICIDS 2017 Network Traffic 6
4.1 Dataset and Preprocessing 6
4.2 Tabular GAN 6
4.3 Evaluation 7
4.4 DiscussionGANs for Generating Ducks in Cybersecurity 8
5. Part 2.3QuickDraw Birthday Cake Sketches 9
5.1 Dataset 9
5.2 DCGAN Training and Progression 9
5.3 Results 10
5.4 Multi-Category Extension 10
6. Summary of Results 10
7. Conclusion and Future Work 11
References 12

1. Introduction

Generative Adversarial Networks (GANs), introduced by Goodfellow et al. (2014), are a class of deep generative models comprising two neural networksa generator and a discriminatortrained through adversarial competition. The generator learns to produce synthetic data samples, while the discriminator learns to distinguish real from generated samples. When training converges, the generator captures the underlying data distribution sufficiently to produce realistic novel samples. Since their introduction, GANs have found applications across computer vision (Karras et al., 2019), medical imaging (Yi et al., 2019), cybersecurity (Lin et al., 2018), and creative domains.

This report presents a two-part investigation. Part 1 builds MLP-based GANs from scratch on synthetic 2D data to develop intuition about adversarial training dynamics, mode collapse, and the impact of architectural choices. Part 2 scales to real-world datasets using Deep Convolutional GANs (DCGANs) for retinal scans and hand-drawn sketches, and a tabular GAN for network intrusion traffic. Each experiment is evaluated quantitatively where possible and discussed with reference to potential improvements. All code was implemented in Python using PyTorch and executed on Google Colab with T4 GPU acceleration.

2. Part 1GANs on Synthetic 2D Data

2.1 Sine Wave GAN

The first experiment trained a GAN to replicate 2D points sampled from y = sin(x) + epsilon, where x ~ Uniform[-pi, pi] and epsilon ~ N(0, 0.1). Both generator and discriminator are two-layer MLPs with 64 hidden units; the generator uses ReLU and the discriminator uses LeakyReLU(0.2) with sigmoid output. Binary cross-entropy (BCE) loss was used with Adam (lr = 0.0002) for 5,000 epochs.

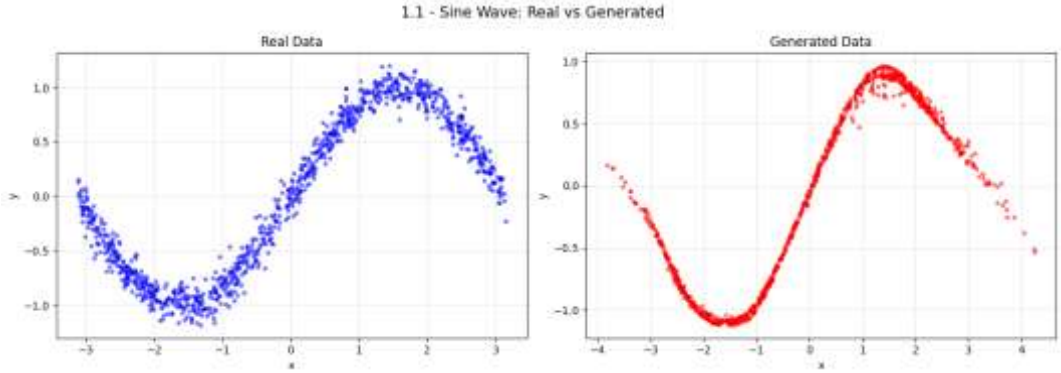


Figure 1: Sine wavereal data (left) vs generated (right). The generator captures the sinusoidal form.

Figure 1 confirms that the generator successfully learned the sinusoidal shape, with density and range closely matching the real data. Slight spread at the extremities (x beyond +/- 3) arises because the generator occasionally maps latent noise outside the training range, a common boundary effect in simple architectures.

2.2 Spiral Distribution GAN

A more challenging target was chosen: a two-arm Archimedean spiral with Gaussian noise (std = 0.03). The architecture was scaled to 128 hidden units and trained for 8,000 epochs to accommodate the increased geometric complexity.

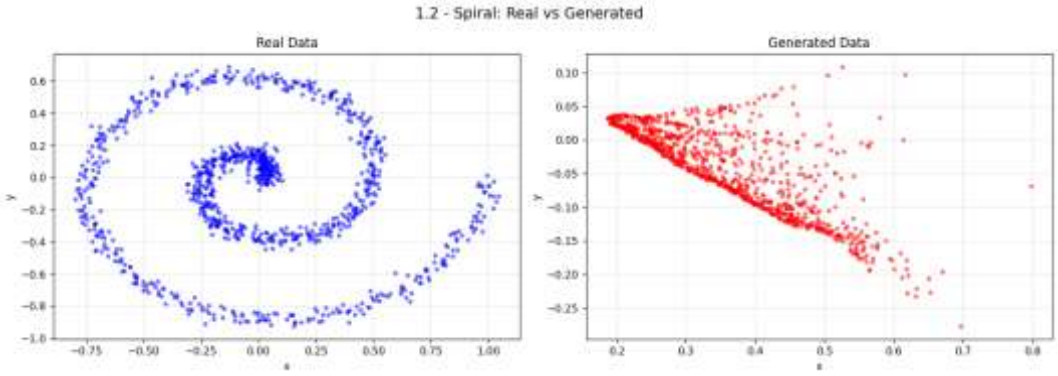


Figure 2: Spiralreal (left) vs generated (right). Partial mode collapse is evident.

Figure 2 reveals **partial mode collapse**the generator learned only one small region of the distribution rather than both spiral arms. This occurs because the spiral is a disconnected, curved manifold; a small MLP with a 2-dimensional latent space cannot easily learn the multi-modal mapping required to cover both arms simultaneously. The generator settled on a single mode where the loss gradient was most favourable. Potential remedies include increasing the latent dimension (e.g. z = 16), applying spectral normalisation to the discriminator (Miyato et al., 2018), or using mini-batch discrimination to encourage diversity.

2.3 Architecture Comparison

Two architectures were compared on the spiral data to study how design choices affect training stability. The original uses ReLU with 2 hidden layers of 64 units. The modified version uses ELU activations, batch normalisation after each hidden layer, dropout (0.3) in the discriminator, and 3 hidden layers of 128 units.

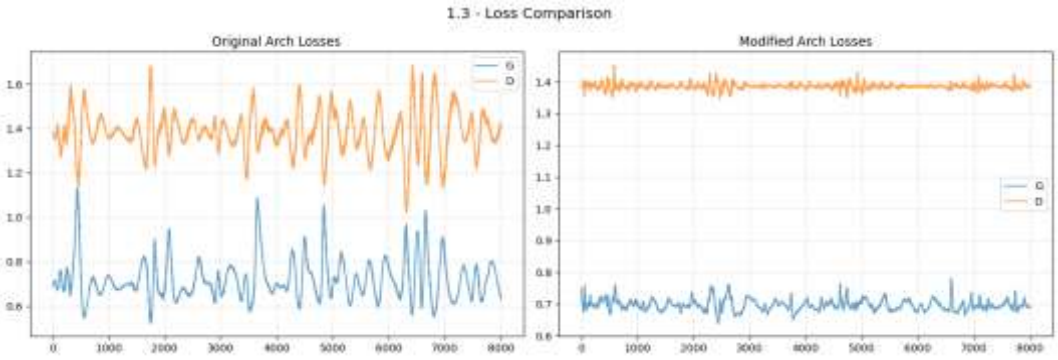


Figure 3: Training lossesoriginal (left) shows high-frequency oscillation; modified (right) is stable.

The loss curves in Figure 3 reveal a stark difference. The original architecture exhibits high-frequency oscillations in discriminator loss, reflecting an unstable adversarial dynamic where D repeatedly overpowers G. The modified architecture converges to a smooth equilibrium near **log(2) ≈ 0.693** for both G and Dthe theoretical Nash equilibrium for BCE-based GANs (Goodfellow et al., 2014). This demonstrates that batch normalisation reduces internal covariate shift, ELU activations avoid dead neurons better than ReLU, and dropout prevents discriminator overfitting. These findings align with the DCGAN guidelines of Radford et al. (2016), which recommend batch normalisation and LeakyReLU as standard practice.

3. Part 2.1OCTMNIST Retinal OCT Images

3.1 Dataset

OCTMNIST (Yang et al., 2023) is part of the MedMNIST benchmark, containing 28x28 grayscale optical coherence tomography (OCT) scans across four retinal conditions: choroidal neovascularisation (CNV), diabetic macular oedema (DME), drusen deposits, and normal retina. The training set contains 97,477 images. Images were normalised to [-1, 1] for Tanh-output compatibility.

3.2 DCGAN Architecture

The DCGAN follows Radford et al. (2016). The generator projects a 100-dimensional latent vector z ~ N(0,1) through a fully connected layer to a 7x7x256 spatial tensor, then applies two transposed convolutions (stride 2, kernel 4) with batch normalisation and ReLU to upsample to 28x28, finishing with a 3x3 convolution and Tanh activation. The discriminator mirrors this with two strided convolutions, LeakyReLU(0.2), and a sigmoid classifier. Weights were initialised from N(0, 0.02). Training used Adam (lr = 0.0002, beta1 = 0.5) for 50 epochs with batch size 128.

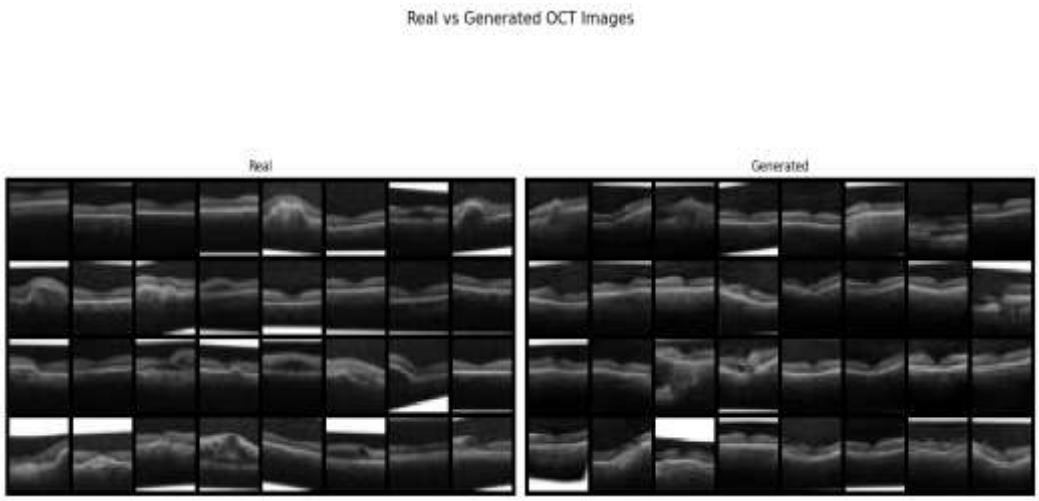


Figure 4: Real OCT images (left) vs DCGAN-generated images (right).

Figure 4 shows that the DCGAN generates images with recognisable retinal layer structuresthe characteristic bright-dark banding of OCT scans is captured. Some images exhibit minor blurring, which is typical of GANs operating at low resolution where fine-grained texture detail is limited by the small spatial dimensions.

3.3 FID Evaluation

The Frechet Inception Distance (FID) was computed using 1,000 real and 1,000 generated images with InceptionV3 features. The obtained FID of **40.39** indicates moderate distributional similarity. For context, state-of-the-art unconditional GANs on CIFAR-10 achieve FID scores below 5 (Karras et al., 2019), but those use much deeper architectures (StyleGAN, Progressive GAN) with extensive training. An FID of 40 for a basic DCGAN on small medical images is a reasonable baseline. Lower scores could be achieved by increasing training epochs, using WGAN-GP loss (Gulrajani et al., 2017), or implementing progressive growing.

3.4 ExtensionConditional GAN

A conditional GAN (cGAN) was implemented following Mirza and Osindero (2014). Class labels were embedded into 10-dimensional vectors and concatenated with z for the generator; for the discriminator, labels were projected to a 28x28 spatial map and concatenated as an additional input channel.



Figure 5: cGAN lossesgenerator loss climbs to 25 while discriminator collapses to 0.

The cGAN suffered **complete training collapse**. As shown in Figure 5, the discriminator rapidly achieved near-perfect classification (D loss → 0), causing the generator to receive vanishing gradients and its loss to increase unboundedly. This is a well-documented failure mode (Salimans et al., 2016). The additional label information made the discriminator's task easier, amplifying the power imbalance. Remedies include: (1) one-sided label smoothing (real labels = 0.9 instead of 1.0), (2) adding Gaussian noise to discriminator inputs that decays over training, (3) replacing BCE with Wasserstein loss and gradient penalty (Gulrajani et al., 2017), and (4) training the generator for multiple steps per discriminator step.

3.5 DiscussionGANs for Generating Ducks

Applying GANs to generate realistic duck images would follow a similar DCGAN pipeline but with substantially greater complexity. A suitable dataset could be sourced from the CUB-200-2011 bird dataset (Wah et al., 2011) or filtered subsets of ImageNet. The key challenges compared to 28x28 OCT scans include higher resolution (at least 128x128 for recognisable birds), greater colour variety (RGB vs grayscale), and complex pose variation. StyleGAN2 (Karras et al., 2020) or diffusion models would be more appropriate architectures. Conditional generation with species labels could enable targeted duck breed synthesis. Applications could include data augmentation for wildlife monitoring classifiers, where certain species are underrepresented in training data.

4. Part 2.2CICIDS 2017 Network Traffic

4.1 Dataset and Preprocessing

The CICIDS 2017 dataset (Sharafaldin et al., 2018) contains labelled network flows captured over five days simulating real-world attack scenarios. The Wednesday file was used, yielding 691,406 flows across 78 numeric features after preprocessing. The class distribution was heavily imbalanced: BENIGN (439,683), DoS Hulk (230,124), DoS GoldenEye (10,293), DoS slowloris (5,796), DoS Slowhttptest (5,499), and Heartbleed (11). Preprocessing included replacing infinite values, dropping NaN rows, and applying StandardScaler normalisation.

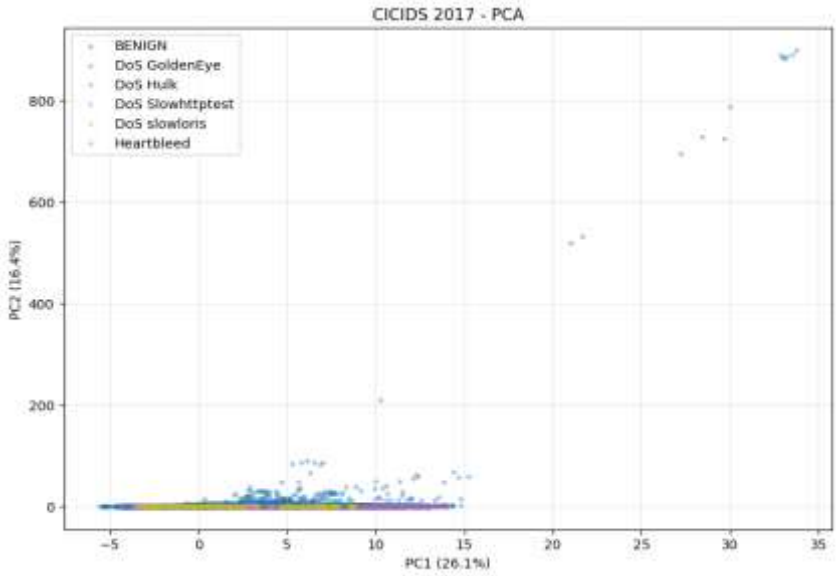


Figure 6: PCA of CICIDS Wednesday showing BENIGN traffic and five DoS attack variants.

4.2 Tabular GAN

Since network traffic features are tabular rather than spatial, an MLP-based GAN was used. The generator maps a 64-dimensional noise vector through three hidden layers (256-512-256) with batch normalisation and ReLU. The discriminator uses LeakyReLU and dropout (0.3). Training ran for 500 epochs with batch size 512 and Adam (lr = 0.0002, beta1 = 0.5).

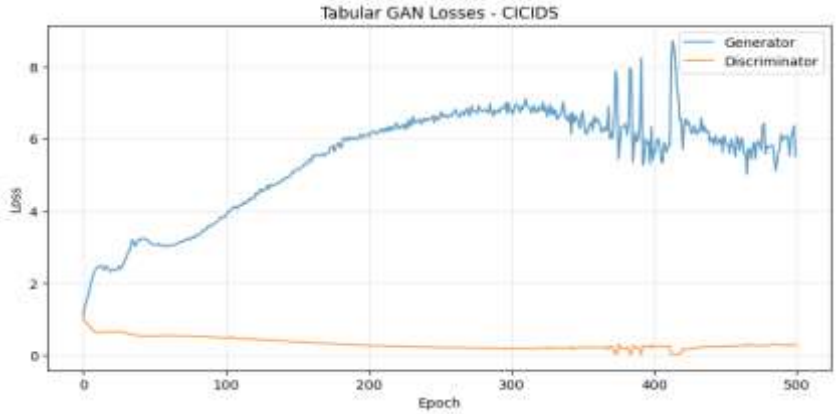


Figure 7: Tabular GAN losses. Generator loss rises while discriminator dominatespartial convergence.

4.3 Evaluation

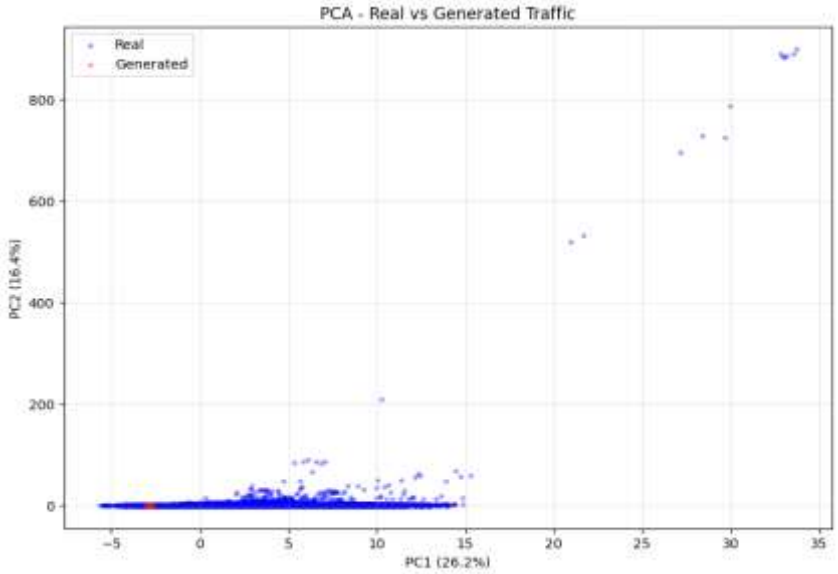


Figure 8: PCA overlayreal (blue) vs generated (red). Generated samples cluster near the origin.

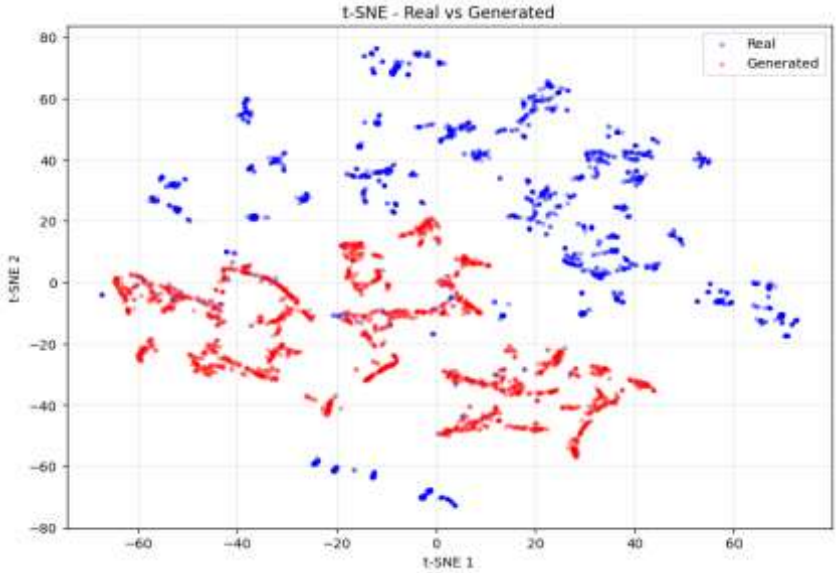


Figure 9: t-SNE comparison showing partial overlap but distinct separation.

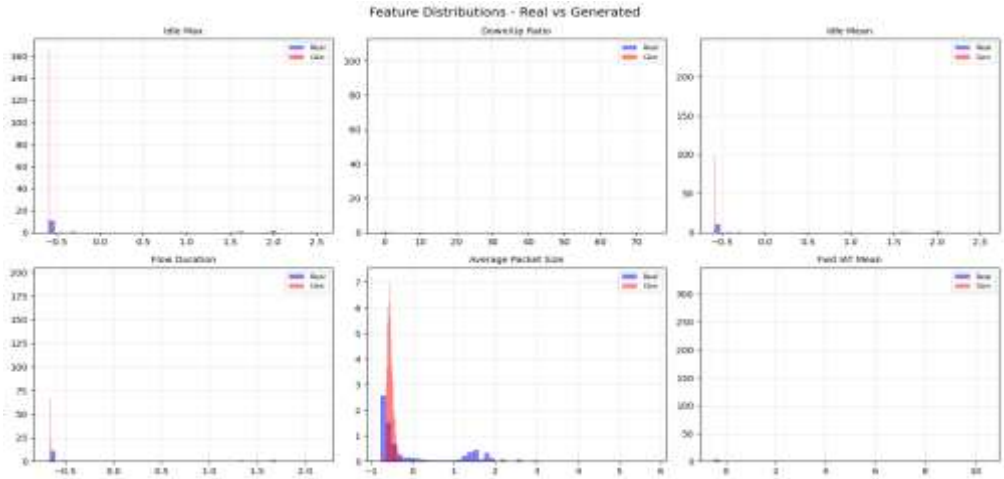


Figure 10: Per-feature histogram overlays for the six highest-variance features.

The evaluation reveals mixed results. PCA (Figure 8) shows the generated data clustered tightly near the origin, failing to capture the wide spread of real outlier traffic. The t-SNE plot (Figure 9) reveals some structural overlap but also clear separation between real and synthetic clusters. Feature-level histograms (Figure 10) show that some features (e.g., Average Packet Size) are modelled reasonably well, while others show significant distributional mismatch. Quantitatively, the mean MAE between real and generated feature means was **0.294**, and the Pearson correlation of feature means was **0.153**, indicating limited fidelity. These results suggest the tabular GAN partially learned the dominant traffic modes but failed to capture the full distributional complexity, particularly heavy-tailed features and rare attack types (e.g., Heartbleed with only 11 samples).

Improvements should focus on architecture and loss function. CTGAN (Xu et al., 2019), specifically designed for tabular data, uses mode-specific normalisation and a conditional generator to handle mixed feature types. Wasserstein loss with gradient penalty would provide more informative gradients than BCE, preventing the discriminator dominance observed. Class-conditional training or oversampling rare attack types could address the severe class imbalance.

4.4 DiscussionGANs for Generating Ducks in Cybersecurity

The concept of using GANs to "generate ducks" connects intriguingly with cybersecurity through the decoy paradigm. Honeypot systems use synthetic network services to attract and study attackers. GANs could generate realistic-looking synthetic traffic to populate these honeypots, making them indistinguishable from production networks. The synthetic traffic must be statistically plausible (passing the same distributional tests our PCA and t-SNE apply) while containing no real sensitive dataessentially "rubber duck" traffic. This is an active research area; Ring et al. (2019) survey the use of synthetic data for intrusion detection, noting that GAN-generated traffic could augment imbalanced datasets like CICIDS where attack classes are severely underrepresented.

5. Part 2.3QuickDraw Birthday Cake Sketches

5.1 Dataset

The QuickDraw dataset (Ha and Eck, 2018) contains millions of hand-drawn sketches from an online game. The "birthday cake" category contains 144,982 simplified 28x28 grayscale bitmaps. A random subset of 50,000 images was used for training. The binary, sketch-like nature of these images (predominantly black background with white strokes) differs substantially from the continuous-tone OCT images, providing a useful architectural stress test.

5.2 DCGAN Training and Progression

The same DCGAN architecture from Part 2.1 was reused, as both datasets are 28x28 single-channel images. Training ran for 50 epochs with identical hyperparameters.



Figure 11: Training progressionfrom noise (epoch 1) to recognisable cakes (epoch 50).

Figure 11 demonstrates clear visual improvement across training. Early epochs produce amorphous blobs; by epoch 20, rectangular cake bases emerge; by epoch 50, candles and layered decorations are visible. This progression confirms the generator is learning hierarchical featureslow-frequency structure first, then finer details.

5.3 Results

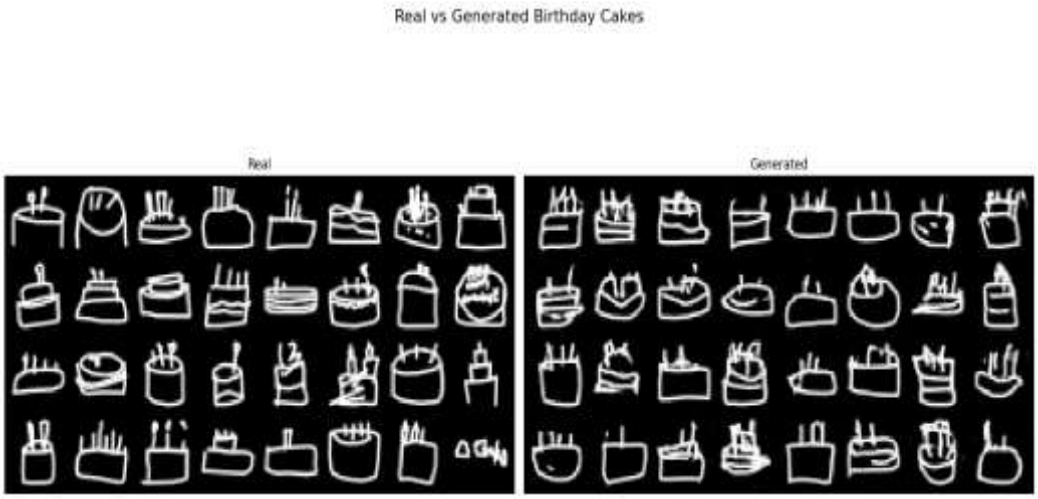


Figure 12: Real (left) vs generated (right) birthday cake sketches.

Figure 12 shows the generator produces recognisable birthday cake sketches with rectangular/rounded bases and candle-like protrusions. The generated sketches match the thick-stroked, simplified aesthetic of QuickDraw drawings. Some mode collapse is visiblea subset of generated cakes share similar rectangular shapesbut overall variety is acceptable for a basic DCGAN.

5.4 Multi-Category Extension

Separate DCGANs were trained on three additional categoriescat, house, treefor 30 epochs each to test cross-category generalisation.

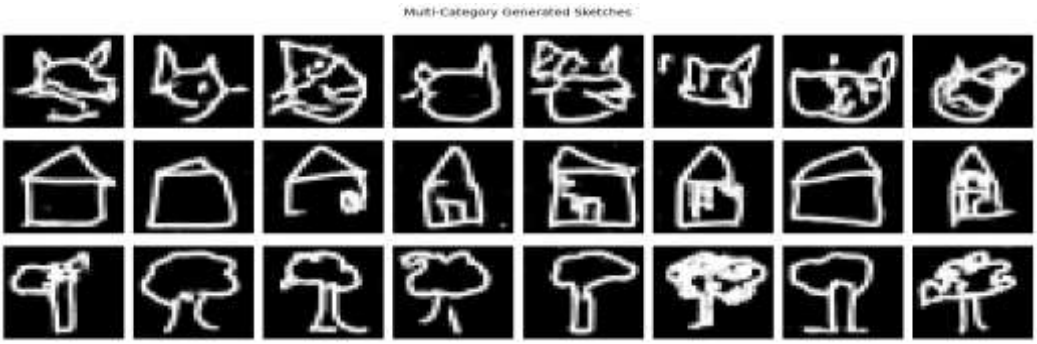


Figure 13: Multi-category generated sketchescat (top), house (middle), tree (bottom).

All three categories produce recognisable outputs. Houses are the most consistent (triangular roofs, rectangular bodies), reflecting their standardised geometric structure. Cats show pointed ears and facial features but with more variation. Trees display trunk-and-canopy structures. This confirms the DCGAN architecture generalises across QuickDraw categories without hyperparameter modification.

6. Summary of Results

Task	Dataset	Key Metric	Outcome
1.1 Sine GAN	Synthetic	Visual match	Excellent
1.2 Spiral GAN	Synthetic	Visual match	Partial mode collapse
1.3 Arch. Comparison	Synthetic	Loss stability	Modified far more stable
2.1 OCTMNIST DCGAN	97,477 images	FID = 40.39	Realistic OCT scans
2.1 cGAN (ext.)	97,477 images	G loss → 25	Training collapse
2.2 CICIDS Tabular	691,406 flows	Corr. = 0.153	Partial fidelity

2.3 QuickDraw DCGAN	144,982 sketches	Visual quality	Recognisable cakes
2.3 Multi-cat (ext.)	3 categories	Visual quality	All recognisable

Table 1: Summary of all experiments and outcomes.

7. Conclusion and Future Work

This study explored GANs across four data domainssynthetic 2D distributions, medical retinal scans, network traffic, and hand-drawn sketches. Part 1 demonstrated that even simple MLPs can learn low-dimensional data distributions, while highlighting the challenge of mode collapse on complex manifolds like spirals. The architecture comparison confirmed that batch normalisation, appropriate activations, and regularisation are essential for stable adversarial training.

Part 2 showed that DCGANs produce visually plausible results on both medical and sketch images, with an FID of 40.39 on OCTMNIST and recognisable birthday cake generation on QuickDraw. The tabular GAN for CICIDS achieved partial distributional coverage but struggled with high-variance and heavy-tailed features, reflecting a known limitation of vanilla GANs on tabular data. The cGAN extension failed due to discriminator dominance, illustrating the sensitivity of conditional architectures to hyperparameter choices.

Future directions include implementing WGAN-GP (Gulrajani et al., 2017) for stabler training across all tasks, using CTGAN (Xu et al., 2019) for tabular data, applying progressive growing (Karras et al., 2018) for higher-resolution image synthesis, and exploring diffusion models as an increasingly popular alternative to adversarial training. Ethically, synthetic data generation raises considerations around data privacy (generated samples must not leak training data) and potential misuse (realistic fake medical images or network traffic), which warrant careful governance frameworks as these technologies mature.

References

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) Generative Adversarial Nets. Advances in Neural Information Processing Systems, 27.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V. and Courville, A. (2017) Improved Training of Wasserstein GANs. Advances in Neural Information Processing Systems, 30.

Ha, D. and Eck, D. (2018) A Neural Representation of Sketch Drawings. Proceedings of ICLR.

Karras, T., Aila, T., Laine, S. and Lehtinen, J. (2018) Progressive Growing of GANs for Improved Quality, Stability, and Variation. Proceedings of ICLR.

Karras, T., Laine, S. and Aila, T. (2019) A Style-Based Generator Architecture for Generative Adversarial Networks. Proceedings of CVPR, pp. 4401-4410.

Karras, T., Laine, S., Aittala, M., Hellsten, J., Lehtinen, J. and Aila, T. (2020) Analyzing and Improving the Image Quality of StyleGAN. Proceedings of CVPR.

Lin, Z., Shi, Y. and Xue, Z. (2018) IDSGAN: Generative Adversarial Networks for Attack Generation against Intrusion Detection. arXiv preprint arXiv:1809.02077.

Mirza, M. and Osindero, S. (2014) Conditional Generative Adversarial Nets. arXiv preprint arXiv:1411.1784.

Miyato, T., Kataoka, T., Koyama, M. and Yoshida, Y. (2018) Spectral Normalization for Generative Adversarial Networks. Proceedings of ICLR.

Radford, A., Metz, L. and Chintala, S. (2016) Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. Proceedings of ICLR.

Ring, M., Wunderlich, S., Scheuring, D., Landes, D. and Hotho, A. (2019) A Survey of Network-Based Intrusion Detection Data Sets. Computers & Security, 86, pp. 147-167.

Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A. and Chen, X. (2016) Improved Techniques for Training GANs. Advances in Neural Information Processing Systems, 29.

Sharafaldin, I., Habibi Lashkari, A. and Ghorbani, A.A. (2018) Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. Proceedings of ICISSP.

Wah, C., Branson, S., Welinder, P., Perona, P. and Belongie, S. (2011) The Caltech-UCSD Birds-200-2011 Dataset. Technical Report CNS-TR-2011-001.

Xu, L., Skoularidou, M., Cuesta-Infante, A. and Veeramachaneni, K. (2019) Modeling Tabular Data using Conditional GAN. Advances in Neural Information Processing Systems, 32.

Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Pfister, H. and Ni, B. (2023) MedMNIST v2: A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification. Scientific Data, 10(1), p.41.

Yi, X., Walia, E. and Babyn, P. (2019) Generative Adversarial Network in Medical Imaging: A Review. Medical Image Analysis, 58, 101552.